

Automated Detection of Olive Fly (*Bactrocera oleae*) Using Color-Space Segmentation and a Multilayer Perceptron Neural Network

Aleyna Musa, Ege Ramadanov, Georgi Markov

Abstract

The olive fly (*Bactrocera oleae*) is among the most economically damaging pests in olive cultivation, causing significant losses in both fruit yield and oil quality. This paper presents an automated, image-based detection system that classifies photographs of adhesive monitoring traps as containing or not containing an olive fly. The pipeline consists of three sequential stages: foreground segmentation via Otsu thresholding applied to the b^* channel of the CIELAB color space, extraction of 13 handcrafted morphological and color features, and binary classification using a compact Multilayer Perceptron (MLP) neural network. Class imbalance in the dataset (1:6.5 ratio) was addressed through geometric and photometric data augmentation applied to the minority class. A decision threshold was selected to maximize the F1 score rather than accuracy, reflecting the cost asymmetry of missed detections. On the full labeled training set the system achieved 89.83% accuracy, 60.61% precision, and 68.89% recall at the optimal threshold of 0.70. The inference pipeline is implemented using only NumPy and OpenCV, making it suitable for deployment on resource-constrained embedded hardware.

Keywords: olive fly detection, Bactrocera oleae, CIELAB color space, morphological feature extraction, multilayer perceptron, binary image classification

1. Introduction

The olive fly (*Bactrocera oleae*, Rossi) is a dipteran pest endemic to the Mediterranean basin and now widespread in olive-growing regions across Europe, the Middle East, Africa, and California. Female flies oviposit directly into developing olive drupes; the hatching larvae feed on the pulp, triggering premature fruit drop and dramatically elevating free-fatty-acid levels in the resulting oil. Economic losses attributable to olive fly infestations routinely exceed 20–30% of annual yield in affected groves and can render entire harvests commercially unviable (Daane & Johnson, 2010).

Integrated pest management (IPM) protocols rely on population monitoring through sticky yellow chromotropic traps hung in the canopy. Inspecting these traps requires trained agronomists to physically visit each location on a regular schedule — typically weekly — count the captured insects, and distinguish olive flies from other species adhering to the surface. This process is labor-intensive, inconsistent across observers, and difficult to scale to large plantations.

Automating trap inspection through computer vision offers a practical alternative: cameras attached to the traps capture images at fixed intervals, and an on-device model classifies each image without human intervention. Crucially, any such model must run on low-power hardware (e.g., a Raspberry Pi) with no dependency on cloud connectivity or large machine-learning frameworks. These hardware constraints motivate the design choices described in this paper.

The task is formulated as a supervised binary classification problem: given a trap image, predict whether it contains at least one olive fly. The positive class is denoted `olive_fly` and the negative class `not_olive_fly`.

2. Dataset

The dataset consists of .JPG images captured from adhesive olive fly traps and organized into two directories:

- `olive_fly/` — 630 images, each containing at least one identifiable olive fly specimen.
- `not_olive_fly/` — 4,070 images of trap contents without olive flies (other insects, debris, empty sections).

This yields a class imbalance ratio of approximately 1:6.5 in favor of the negative class. A classifier trained on this distribution without correction would be strongly biased toward predicting the majority class, achieving high nominal accuracy while failing to detect any true positives. The strategy used to address this imbalance is described in Section 4.

All images are taken under natural outdoor lighting conditions with variable illumination, shadow, and camera angle, making the detection task non-trivial. The traps' saturated yellow adhesive backing is a consistent and exploitable feature that anchors the segmentation approach.

3. Methodology

3.1 Foreground Segmentation

The goal of segmentation is to isolate the foreground objects — insects, debris, or empty regions — from the uniform yellow trap background. Performing feature extraction on the raw image would conflate background pixels with object pixels and introduce noise.

Color Space Conversion. Images are converted from BGR to the CIELAB color space. CIELAB was designed to be perceptually uniform and separates luminance (L^*) from color information (a^* and b^*). The b^* channel encodes the blue–yellow opponent axis: the yellow trap background produces consistently high b^* values, whereas darker foreground objects (flies, other insects, organic debris) produce distinctly lower b^* values. Thresholding on b^* therefore separates foreground from background far more robustly than a grayscale luminance threshold, which conflates objects with similarly bright backgrounds and is sensitive to lighting variation and shadow.

Otsu's Thresholding. Rather than setting a fixed b^* threshold manually, Otsu's method is applied automatically (Otsu, 1979). This algorithm exhaustively searches for the threshold value that minimizes the weighted intra-class variance of the two resulting pixel groups, effectively maximizing the between-class separation. Otsu's method adapts automatically to the distribution of each individual image, making it resilient to global exposure differences across the dataset.

Morphological Cleaning. The raw binary mask produced by thresholding typically contains spurious noise pixels (isolated foreground specks from dust or trap artifacts) and small holes within genuine foreground regions. Two morphological operations are applied in sequence to clean the mask (Gonzalez & Woods, 2018): morphological opening (erosion followed by dilation, using a 7×7 rectangular structuring element) removes isolated noise pixels;

morphological closing (dilation followed by erosion, same kernel) fills small internal holes. The kernel size of 7×7 was chosen empirically to remove sub-pixel noise while preserving the fine leg and wing structures of the fly.

Connected Component Analysis. After morphological cleaning, the mask may still contain multiple disconnected blobs corresponding to different objects or residual noise. Each connected region is labeled with a unique integer identifier, and the single largest region by pixel area is retained as the final foreground mask. This ensures that subsequent feature extraction describes a single coherent object rather than a fragmented set of regions.

3.2 Data Augmentation

To counteract the 1:6.5 class imbalance, synthetic positive-class samples are generated from existing olive_fly images through random augmentation until the minority class matches the majority class in size (Shorten & Khoshgoftaar, 2019). Four stochastic operations are applied independently during each augmentation:

- **Horizontal flip** with 50% probability.
- **Vertical flip** with 50% probability.
- **Affine transformation:** random rotation uniformly sampled from $[-30^\circ, +30^\circ]$ and random scale uniformly sampled from $[0.9, 1.1]$, with reflected border padding to avoid black border artifacts.
- **Photometric jitter:** pixel values multiplied by a brightness factor in $[0.85, 1.15]$ and shifted by a contrast offset in $[-25, +25]$, then clamped to $[0, 255]$.

All operations use a fixed random seed of 42 to ensure reproducibility. The augmented images are used exclusively for training and are excluded from evaluation.

3.3 Feature Extraction

After segmentation, 13 numerical features are computed from each image. The features are divided into two groups: shape descriptors derived from the binary mask geometry, and color descriptors computed over the pixels inside the foreground mask.

Shape Features (Table 1)

#	Feature	Description
1	area_frac	Foreground pixels / total pixels — relative object size
2	aspect	Width / height of the bounding rectangle
3	solidity	Contour area / convex hull area — compactness measure
4	circularity	$4\pi \cdot \text{area} / \text{perimeter}^2$ — closeness to a circle (clipped at 1.5)
5	eccentricity	Major axis / minor axis of the fitted ellipse — elongation (clipped at 5.0)
6	hu1	Log-transformed first Hu moment (Hu, 1962) — rotation- and scale-invariant shape descriptor

7	ncomp	Count of connected components whose area exceeds 10% of total foreground
---	-------	--

Color Features (Table 2)

#	Feature	Description
8	hue	Mean HSV hue over foreground pixels
9	saturation	Mean HSV saturation over foreground pixels
10	value	Mean HSV value (brightness) over foreground pixels
11	L_mean	Mean CIELAB L* (luminance) over foreground pixels
12	a_mean	Mean CIELAB a* (green–red) over foreground pixels
13	b_mean	Mean CIELAB b* (blue–yellow) over foreground pixels

3.4 Feature Normalization

Raw feature values span heterogeneous numerical ranges — area_frac is bounded in [0, 1] while hue spans [0, 180]. Without normalization, high-magnitude features would dominate gradient updates during neural network training, degrading convergence. Each feature is z-score standardized:

$$x_{scaled} = (x - \mu) / \sigma$$

where μ and σ are computed per feature across the full balanced (augmented) training set. The same μ and σ are applied at inference time, so they are stored alongside the model weights.

3.5 Neural Network Architecture and Training

A Multilayer Perceptron (MLP) is used for binary classification. The architecture is deliberately compact to keep inference fast and the model transferable to embedded hardware:

- **Input layer:** 13 neurons (one per standardized feature).
- **Hidden layer:** 10 neurons with Rectified Linear Unit (ReLU) activation, $f(x) = \max(0, x)$.
- **Output layer:** 1 neuron with sigmoid activation, producing a probability $p \in [0, 1]$ that the image belongs to the olive_fly class.

The model is trained using scikit-learn’s `MLPClassifier` (Pedregosa et al., 2011) with a maximum of 8,000 iterations.

Decision Threshold Tuning. The default classification threshold of 0.50 is not optimal when the cost of a missed detection (false negative) substantially outweighs the cost of a false alarm (false

positive). The threshold is therefore swept over 81 equally spaced values in $[0.10, 0.90]$ on the real (non-augmented) training images, and the value that maximizes the F1 score is selected. The optimal threshold was **0.70**, reflecting a conservative stance that reduces false positives at the expense of some additional false negatives.

4. Inference Pipeline

At deployment, the trained model weights (W_1, B_1, W_2, B_2) and normalization statistics (μ, σ) are extracted from scikit-learn and hard-coded into a self-contained `detect_olive_fly` function implemented in plain NumPy and OpenCV. This eliminates runtime dependencies on scikit-learn or any deep-learning framework and allows the model to run on a Raspberry Pi without any model serialization files. The forward pass is:

- $h = \text{ReLU}(X \cdot W_1 + B_1)$
- $p = \sigma(h \cdot W_2 + B_2)$
- Return `olive_fly` if $p \geq 0.70$, else `not_olive_fly`.

5. Results

The system is evaluated on the full labeled training dataset ($n = 2,350$ real images, excluding all augmented samples) using the tuned threshold of 0.70.

Table 3: Confusion Matrix ($n = 2,350$)

	Predicted positive	Predicted negative
Actual positive	TP = 217	FN = 98
Actual negative	FP = 141	TN = 1,894

The resulting performance metrics are:

Metric	Value
Accuracy	$(217 + 1,894) / 2,350 = 89.83\%$
Precision	$217 / (217 + 141) = 60.61\%$
Recall	$217 / (217 + 98) = 68.89\%$
F1 Score	$2 \times (0.6061 \times 0.6889) / (0.6061 + 0.6889) = 64.50\%$

The high overall accuracy (89.83%) is in part explained by the class imbalance: the negative class constitutes 84% of the test images, so a trivial majority-class predictor would already reach 84% accuracy. The more informative metrics are precision and recall. A recall of 68.89% means the system detects approximately 7 in 10 olive flies present in the traps, and a precision of 60.61% means roughly 6 in 10 positive predictions are true detections. These values indicate that the system is a useful decision-support tool while leaving room for improvement, particularly in reducing false negatives.

6. Discussion

The primary limitation of the current approach is its reliance on handcrafted features. The 13 features were selected through domain knowledge and manual experimentation; a more systematic feature selection or a learned representation (e.g., from a convolutional neural network) might capture discriminative patterns that the handcrafted set misses. In particular, the system struggles when debris or other insects share the dark-body morphology of the olive fly, which the color and shape features alone cannot fully resolve.

The choice to threshold on CIELAB b^* rather than HSV or raw grayscale is well-motivated: the yellow trap background is highly saturated in the yellow direction, producing a large separation in b^* from dark foreground objects regardless of scene illumination. However, the current approach retains only the single largest connected component as the foreground, which will fail when multiple olive flies are present in the same image — an important edge case for heavily infested traps.

The compact MLP architecture ($13 \rightarrow 10 \rightarrow 1$) was deliberately chosen for embedded deployment. Evaluation on a held-out test set not seen during training or threshold tuning would provide a less optimistic and more trustworthy estimate of generalization performance — this is the most important step for future work.

7. Conclusion

This paper presented an automated pipeline for detecting olive fly (*Bactrocera oleae*) in adhesive trap images. The system combines CIELAB b^* segmentation with Otsu thresholding, 13 handcrafted morphological and color features, and a compact MLP classifier tuned to maximize F1. The entire inference path runs on NumPy and OpenCV, requiring no heavy ML framework at runtime and making the system deployable on low-cost embedded hardware. On the labeled training dataset the pipeline achieved 89.83% accuracy, 60.61% precision, and 68.89% recall. Future work should evaluate performance on a true held-out test set, explore multi-fly detection, and investigate whether learned convolutional features can improve recall without sacrificing the deployment simplicity.

References

- Daane, K. M., & Johnson, M. W. (2010). Olive fruit fly: Managing an ancient pest in modern times. *Annual Review of Entomology*, 55, 291–310. <https://doi.org/10.1146/annurev.ento.54.110807.090553>
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.
- Hu, M.-K. (1962). Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, 8(2), 179–187. <https://doi.org/10.1109/TIT.1962.1057692>
- Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66. <https://doi.org/10.1109/TSMC.1979.4310076>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), Article 60. <https://doi.org/10.1186/s40537-019-0197-0>